PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 08 -

CRIANDO INFRAESTRUTURA COMPLETA NA AWS

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 6
MERCADO, CASES E TENDÊNCIAS 15
O QUE VOCÊ VIU NESTA AULA? 16
REFERÊNCIAS 17

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Depois de explorar os fundamentos da Infraestrutura como Código, aprender a modularizar projetos, gerenciar ambientes com Terragrunt e integrar IaC a pipelines de automação, você está pronto(a) para o **grande desafio final**: **criar uma infraestrutura completa na AWS** aplicando tudo o que viu até agora, de forma prática, integrada e realista.

Este é o momento em que o conhecimento deixa de ser apenas teórico. Agora, o foco é pensar e agir como um **engenheiro de nuvem profissional** — alguém que não apenas escreve código, mas **projeta sistemas**, toma decisões arquiteturais conscientes e sabe como transformar linhas de código em uma base de nuvem segura, automatizada e sustentável.

O desafio desta aula representa o que acontece em **ambientes reais de empresas que adotam IaC**: planejar, desenhar, codificar e automatizar uma infraestrutura do zero, mantendo boas práticas de versionamento, segurança e governança.

Ao final, você terá construído algo muito próximo do que uma equipe DevOps de verdade entrega diariamente — uma arquitetura funcional, escalável e reproduzível, pronta para hospedar aplicações de produção.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Nesta etapa, vamos construir, versionar e automatizar uma **infraestrutura completa na AWS**. A proposta é criar uma estrutura de nuvem moderna, escalável e segura — tudo 100% codificado com **Terraform ou OpenTofu**, seguindo o padrão visto nas aulas anteriores.

Vamos desenvolver um projeto modular e multiambiente (Dev, Staging, Prod), contendo rede, computação, armazenamento e backend remoto, integrados a um pipeline automatizado via **GitHub Actions**.

Escopo

Rede (VPC e sub-redes)

1. Crie uma **VPC** com sub-redes públicas e privadas em duas zonas de disponibilidade.
2. Configure tabelas de rotas, gateways de internet e NAT gateways.
3. Configure grupos de segurança para controlar o tráfego entre camadas.

Camada de computação (EC2 ou EKS)

4. Configure pelo menos uma instância **EC2** (ou um cluster **EKS**) em sub-redes privadas.
5. Use variáveis e módulos para facilitar a reutilização e o escalonamento.

Camada de dados e armazenamento

6. Crie um banco de dados **RDS (PostgreSQL)** e um **bucket S3** para armazenamento de arquivos ou logs.
7. Habilite criptografia e defina políticas de acesso restritas.

Backend remoto e versionamento

8. Configure o backend em **S3 + DynamoDB** para armazenar o `terraform.tfstate` e evitar conflitos de concorrência.

Automação (CI/CD)

9. Configure o **GitHub Actions** com autenticação via **OIDC** (sem chaves fixas).

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

10. O pipeline deve executar as etapas `terraform fmt`, `validate`, `plan` e `apply`.

Documentação

11. Inclua um `README.md` explicando a estrutura, comandos e decisões técnicas.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

A construção de uma **infraestrutura completa** é o ápice do aprendizado em Infraestrutura como Código. Nesta etapa, você deixa de ser apenas um executor de comandos e passa a atuar como **um arquiteto de nuvem**, projetando soluções, tomando decisões estruturais e aplicando boas práticas de automação, modularização e segurança.

Ao longo do curso, vimos como o Terraform (ou OpenTofu) se tornou a espinha dorsal de uma cultura de entrega contínua e governança na nuvem. Agora, você vai reunir tudo isso para criar um **projeto real**: uma arquitetura completa na AWS, automatizada com GitHub Actions e estruturada para suportar múltiplos ambientes.

Planejamento e arquitetura

Antes de codificar, é preciso **planejar a arquitetura**. Uma boa arquitetura em nuvem é baseada em **modularidade, isolamento e automação**. Isso inclui entender os componentes que formam a base de qualquer infraestrutura moderna:

- **VPC (Virtual Private Cloud)**: define o isolamento de rede e controle de endereçamento IP.
- **Sub-redes públicas e privadas**: separam camadas de exposição.
- **Gateways e tabelas de rota**: conectam recursos internos à internet.
- **Serviços de computação (EC2 ou EKS)**: executam workloads.
- **Armazenamento (S3 e RDS)**: persistem dados e artefatos.
- **Backend remoto**: guarda o estado do Terraform com segurança e bloqueio de concorrência.
- **Pipelines de automação (CI/CD)**: padronizam deploys e garantem rastreabilidade.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

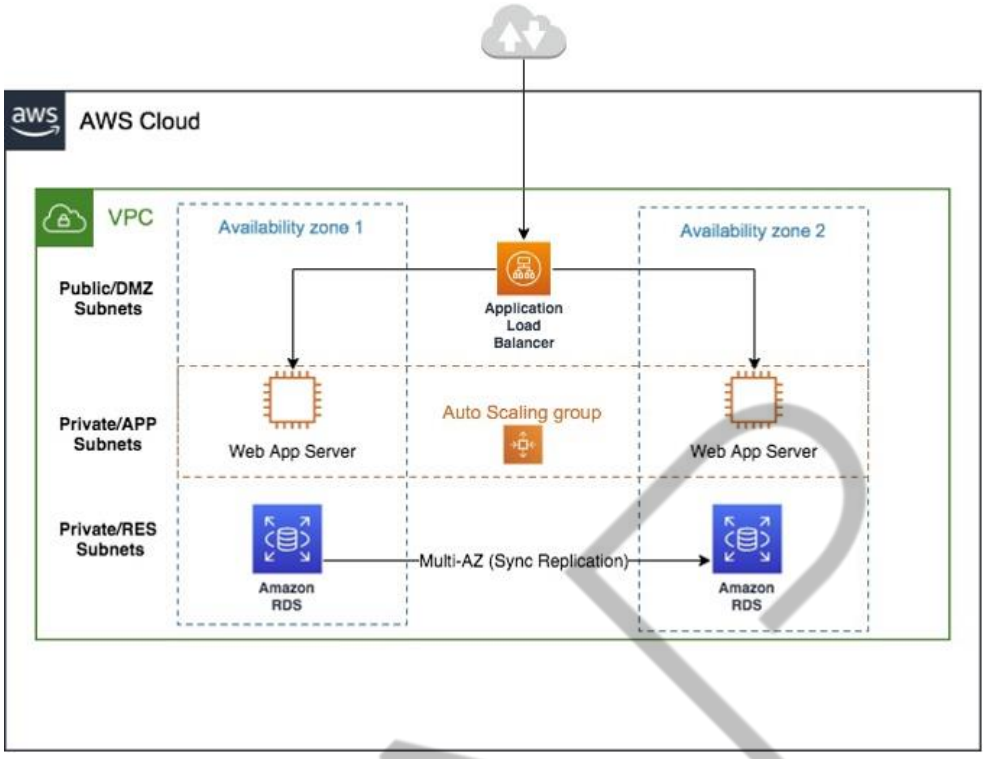


Figura 1 – VPC
Fonte: AWS (2025)

Esses elementos formam o esqueleto de qualquer aplicação moderna em nuvem.

Modularização e Reutilização

Ao criar módulos (como `vpc`, `ec2`, `rds`, `s3`), você está aplicando **engenharia de software na infraestrutura**. Cada módulo representa um "bloco de construção" independente, que pode ser testado, versionado e reutilizado.

Estude os repositórios públicos do **Terraform Registry** e do **TofuHub**. Eles oferecem módulos prontos e mantidos pela comunidade — como **`terraform-aws-modules/vpc/aws` ou `terraform-aws-modules/ec2-instance/aws`** — que você pode reutilizar e adaptar.

Esses repositórios também são um excelente exemplo de padrões de versionamento e documentação, ensinando como publicar seus próprios módulos no futuro.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Gerenciamento de Ambientes e Backends

Cada ambiente (Dev, Staging, Prod) deve ter seu próprio **state** e variáveis personalizadas, mas compartilhar os mesmos módulos.

O backend remoto em **S3 + DynamoDB** garante segurança e consistência, evitando que dois pipelines apliquem alterações simultaneamente.

Terragrunt facilita o controle de múltiplos ambientes e contas AWS. Ele herda variáveis globais e permite executar comandos em cascata com `run-all`. Estudar essa ferramenta é o passo seguinte para quem deseja atuar em **operações corporativas com dezenas de ambientes segregados**, mantendo o princípio DRY (Don't Repeat Yourself).

Segurança e Auditoria

Segurança não é uma etapa: é uma cultura. Em IaC, isso significa definir segurança **desde o código**, com políticas, roles e criptografia.

AWS Well-Architected Framework (Pilar de Segurança)

Esse documento ensina como projetar controles de identidade, proteger dados e automatizar auditoria. Estudá-lo fará você compreender como segurança e conformidade são integradas a cada decisão técnica, não adicionadas depois.



Figura 2 – Pilar de segurança
Fonte: Elaborado pelo autor (2025)

Automação e GitOps

Um projeto IaC só é sustentável se for automatizado. Com pipelines CI/CD e GitOps, toda mudança no código é revisada, validada e aplicada de forma previsível. Isso evita erros manuais, melhora a colaboração e cria um histórico completo de alterações. Explore ferramentas como ArgoCD e FluxCD para automatizar a sincronização entre o repositório Git e a infraestrutura.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Essas plataformas verificam continuamente se o estado real do ambiente corresponde ao código versionado e aplicam correções automáticas se detectarem divergência.

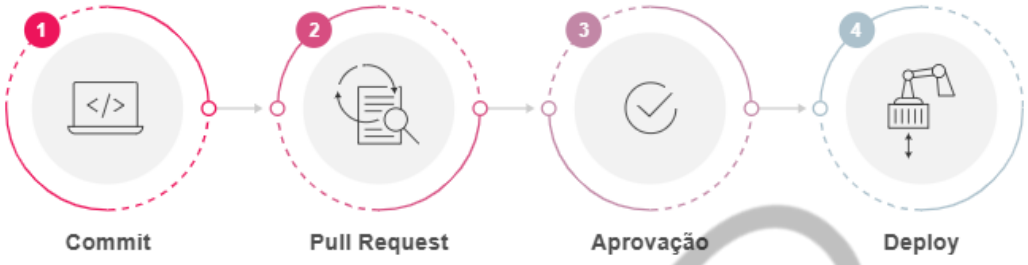


Figura 3 – Pipeline GtOps
Fonte: Elaborado pelo autor (2025)

Estrutura de Diretórios

O primeiro passo para a prática é criar uma estrutura organizada de diretórios. Isso define a separação entre módulos reutilizáveis e ambientes específicos.

```
infra/
├── modules/
│   ├── vpc/
│   ├── ec2/
│   ├── rds/
│   ├── s3/
│   └── iam/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── backend/
│   ├── backend.tf
│   └── dynamodb.tf
└── .github/workflows/
    └── terraform.yaml}
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Essa estrutura reflete o padrão usado em empresas que aplicam IaC em escala. Os módulos são “blocos de construção”, enquanto os ambientes são apenas variações desses blocos. O resultado é previsibilidade e facilidade de manutenção.

Criando a VPC

O primeiro módulo define a base da rede, a **VPC** e suas sub-redes:

```
# modules/vpc/main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.env}-vpc"
  }
}

resource "aws_subnet" "public" {
  count                   = length(var.public_subnets)
  vpc_id                  = aws_vpc.main.id
  cidr_block                 = element(var.public_subnets,
count.index)
  map_public_ip_on_launch = true
  availability_zone       = element(var.azs, count.index)
  tags = {
    Name = "${var.env}-public-${count.index}"
  }
}

resource "aws_subnet" "private" {
  count             = length(var.private_subnets)
  vpc_id            = aws_vpc.main.id
  cidr_block              =  element(var.private_subnets,
count.index)
  availability_zone = element(var.azs, count.index)
  tags = {
    Name = "${var.env}-private-${count.index}"
  }
}
```

Aqui, a VPC é configurada com DNS habilitado e sub-redes públicas e privadas distribuídas entre zonas de disponibilidade. O uso de **count e element()** torna o módulo dinâmico e reutilizável, um conceito-chave em IaC.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Bucket S3 (State e Armazenamento)

```
# modules/s3/main.tf
resource "aws_s3_bucket" "state" {
  bucket = "${var.env}-iac-state"
  force_destroy = true

  versioning { enabled = true }

  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default{
sse_algorithm = "AES256" }
    }
  }

  tags = { Name = "${var.env}-iac-state" }
}
```

Este bucket S3 armazena o **arquivo de state remoto**, permitindo que o time compartilhe e audite mudanças. A criptografia AES-256 garante segurança, e o versionamento fornece histórico automático.

DynamoDB para Lock do State

```
# backend/dynamodb.tf
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute { name = "LockID"; type = "S" }

  tags = { Purpose = "Terraform State Lock" }
}
```

Essa tabela DynamoDB previne alterações simultâneas no state, um problema comum em times colaborativos. Essa abordagem é a base do controle de concorrência no Terraform.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Instância EC2

```
# modules/ec2/main.tf
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = element(var.private_subnets, 0)
  key_name      = var.key_pair

  tags = { Name = "${var.env}-app-instance" }
}
```

Aqui, criamos uma instância EC2 na sub-rede privada, isolando a camada de aplicação da internet. O acesso deve ser feito por bastion host ou AWS Systems Manager (SSM), reforçando boas práticas de segurança, mas também pode ser feito diretamente via SSH.

Banco de Dados RDS

```
# modules/rds/main.tf
resource "aws_db_instance" "main" {
  identifier        = "${var.env}-db"
  engine            = "postgres"
  instance_class    = "db.t3.micro"
  allocated_storage = 20
  username          = var.db_user
  password          = var.db_pass
  skip_final_snapshot = true
  vpc_security_group_ids = [var.db_sg]
  db_subnet_group_name   = var.db_subnet_group
  publicly_accessible    = false

  tags = { Name = "${var.env}-rds" }
}
```

O banco é privado e criptografado, com credenciais passadas por variáveis seguras. Em produção, recomenda-se usar o **AWS Secrets Manager** e snapshots automáticos.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Integração de Módulos

```
# environments/dev/main.tf
terraform {
  backend "s3" {
    bucket         = "dev-iac-state"
    key            = "terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}

module "vpc" {
  source          = "../../modules/vpc"
  vpc_cidr        = "10.0.0.0/16"
  public_subnets  = ["10.0.1.0/24"]
  private_subnets = ["10.0.2.0/24"]
  azs             = ["us-east-1a"]
  env             = "dev"
}

module "ec2" {
  source          = "../../modules/ec2"
  ami_id          = "ami-0c55b159cbfafe1f0"
  instance_type   = "t3.micro"
  private_subnets = module.vpc.private_subnets
  key_pair        = "iac-key"
  env             = "dev"
}
```

Aqui, unimos os módulos em um ambiente funcional. O backend remoto garante que o state do Terraform seja centralizado e protegido.

Workflow do GitHub

```
# .github/workflows/terraform.yaml
name: Terraform CI/CD
on:
  push:
    branches: ["main"]

permissions:
  id-token: write
  contents: read

jobs:
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
      terraform:
        runs-on: ubuntu-latest
        steps:
          - uses: actions/checkout@v4

          - uses: aws-actions/configure-aws-credentials@v4
            with:
              role-to-assume:
arn:aws:iam::<ACCOUNT_ID>:role/github-actions-terraform
              aws-region: us-east-1

          - uses: hashicorp/setup-terraform@v3

          - run: terraform init
          - run: terraform plan
```

Este pipeline autentica via OIDC, uma forma moderna e segura de conceder acesso temporário à AWS. Isso elimina o uso de credenciais fixas e torna o CI/CD mais seguro e rastreável.

Conclusão

Nesta aula, foi realizado o ciclo completo da Infraestrutura como Código, incluindo:

- Planejar arquiteturas seguras e escaláveis;
- Modularizar código para ambientes reutilizáveis;
- Gerenciar estados remotamente;
- Automatizar com pipelines GitOps;
- Documentar para colaboração em equipe.

Agora você tem em mãos **uma base sólida**, com o mesmo tipo de estrutura usada em projetos de engenharia de nuvem reais, capaz de crescer e se adaptar a qualquer demanda.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

Hoje em dia, a adoção de **Infraestrutura como Código (IaC)** não é mais uma escolha opcional — tornou-se um elemento central para organizações que executam workloads em nuvem. A habilidade de versionar, auditar e automatizar infraestrutura permite que times de engenharia escalem, reduzam erros humanos e mantenham governança em ambientes cada vez mais complexos.

A empresa de consultoria ObjectStyle trabalhou com uma **liga esportiva dos EUA** que possuía inúmeras aplicações internas e um sistema multicanal de conteúdo espalhado entre datacenters on-premises e nuvem. Para acelerar a entrega, garantir compliance com dados sensíveis (PII) e implantar novos sites com rapidez, a organização adotou uma abordagem de IaC (utilizando Terraform + Ansible) para codificar a infraestrutura na AWS.

O resultado incluiu a automação de provisão de servidores que antes demorava dias, agora realizada em horas, além da capacidade de **clonar ambientes** (VPC, rede, servidores) em diferentes regiões ou datacenters rapidamente. Esse tipo de transformação mostra como IaC torna-se a fundação de operações ágeis em empresas de escala.

Tendências que se destacam para 2025 em diante

Governança e Policy as Code — A integração de ferramentas como Open Policy Agent (OPA) nos pipelines permite que regras de conformidade sejam aplicadas automaticamente. Isso é cada vez mais relevante em setores regulados (finanças, saúde, telecom).

Organização de código e escalabilidade de repositórios IaC — Artigos como “Organizing Terraform Code for Scalability and Maintainability” trazem técnicas para estruturar projetos grandes com múltiplos módulos, ambientes e contas AWS.

Integração total de DevOps + FinOps + Observabilidade — Infraestrutura como Código já não é só sobre provisionamento: também é sobre **custo, eficiência e visibilidade**. Ferramentas que correlacionam alterações de infra com custos e métricas de performance estão ganhando força.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta última aula, você conectou todos os conceitos estudados ao longo desta disciplina e os aplicou em um desafio completo: **criar uma infraestrutura AWS profissional usando IaC**. Aprendeu como pensar como um arquiteto de nuvem, estruturar módulos, aplicar automação segura e versionar sua infraestrutura.

Agora você é capaz de **planejar, construir e gerenciar ambientes em escala**, aplicando as mesmas práticas utilizadas por engenheiros DevOps em grandes empresas.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

AWS. **Well-Architected Framework.** Disponível em: https://aws.amazon.com/architecture/well-architected/. Acesso em: 06 nov. 2025.

GITHUB. **Using OpenID Connect with GitHub Actions.** Disponível em: https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect. 06 nov. 2025.

HASHICORP. **Terraform**. Disponível em: https://developer.hashicorp.com/terraform. 06 nov. 2025.

TERRAGRUNT. **Terragrunt Documentation.** Disponível em: https://terragrunt.gruntwork.io/docs/. 06 nov. 2025.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. tfsec. checkov, atlantis.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com